

XAI

Основной источник

<https://christophm.github.io/interpretable-ml-book>

XAI

XAI (eXplainable AI) = объяснимый ИИ - набор методов объяснения ответов модели

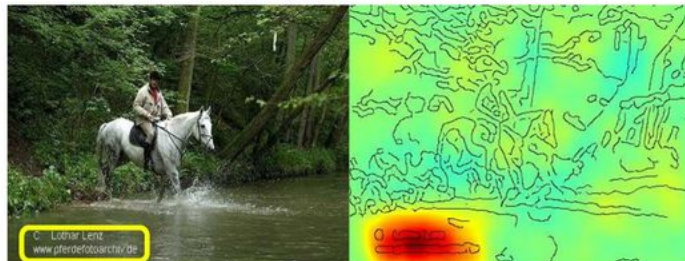
- **explainability** - построение объяснений предсказаний для *произвольных* разработанных моделей
примеры: см. далее в лекции
- **interpretability** - построение моделей, понятных без дополнительных объяснений
примеры: решающие деревья, линейные модели
- **transparency** - предоставление документации
примеры: model cards, dataset datasheets

Зачем нужен XAI?

- дебаг
- оценка модели (напр., EU AI Act)
 - надежность (reliability)
 - справедливость (fairness)
- обеспечение доверия к модели
- извлечение новых знаний

Пример

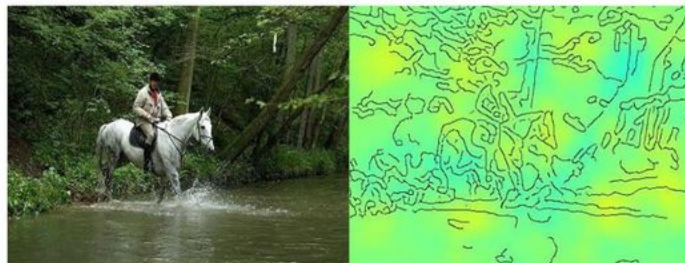
Horse-picture from Pascal VOC data set



Source tag
present



Classified
as horse



No source
tag present



Not classified
as horse

source: <https://arxiv.org/pdf/1902.10178>

Оценка (evaluation) XAI

Свойства объяснения:

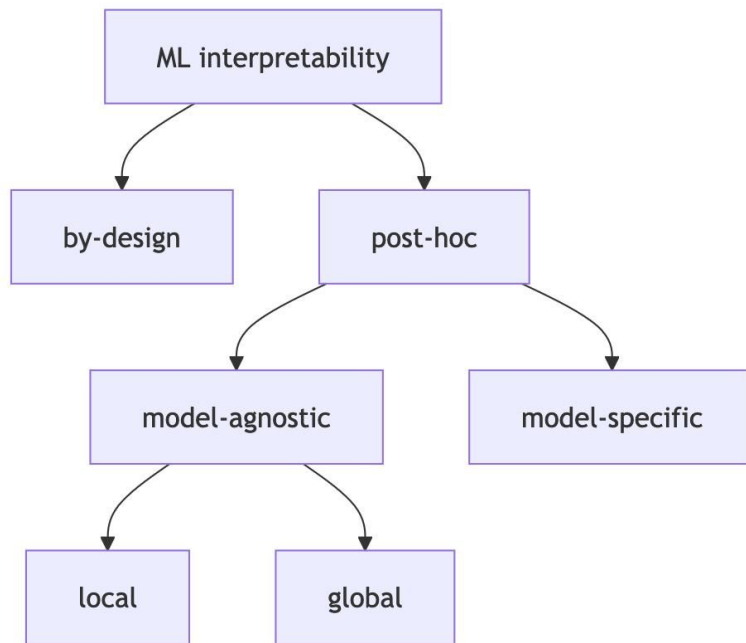
- **Fidelity** - насколько верно объяснение отражает предсказания модели
- **Consistency** - согласованность объяснений разных моделей со схожими предсказаниями
- **Stability** - устойчивость объяснений при небольшом изменении входных данных
- **Comprehensibility** - насколько объяснения понятны человеку

Оценка (evaluation) XAI

Свойства XAI метода:

- **Expressive Power:** пространство допустимых объяснений
Примеры: if-then правила, численные формулы, тексты на естественном языке
- **Translucency:** доступность внутреннего устройства модели при составлении объяснения (black-box vs grey-box vs white-box)
- **Portability:** множество классов моделей, к которым метод можно применить
- **Algorithmic Complexity:** вычислительная сложность метода

Таксономия



source: <https://christophm.github.io/interpretable-ml-book/overview.html>

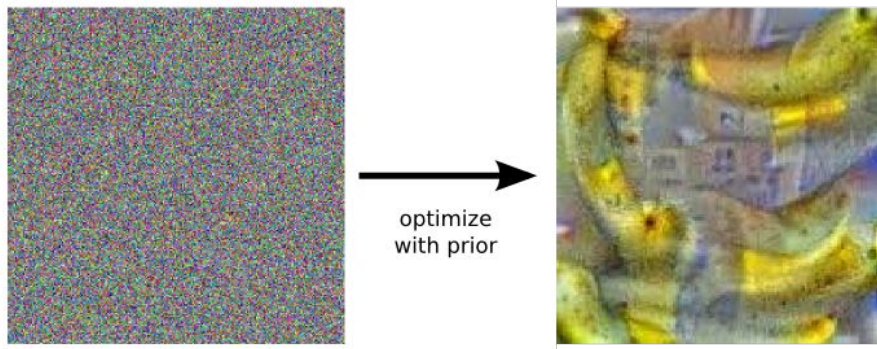
Feature Visualization

Метод существует давно, но стал широко знаменит благодаря DeepDream (2013)

<https://arxiv.org/abs/1312.6034>

Ищем изображение, максимизирующее активацию фичи (нейрона / фильтра / слоя):

- стартуем с случайного шума
- используем Gradient Ascent



Saliency Maps

Карты значимости (saliency maps) оценивают значимость отдельных пикселей для предсказания модели.

Разные названия:

- sensitivity map
- saliency map
- pixel attribution map
- gradient-based attribution methods
- feature relevance
- feature attribution
- feature contribution

Saliency Maps

Обычно gradient-based:

- vanilla gradient
- Grad-CAM

Vanilla gradient

[paper](#)

Алгоритм:

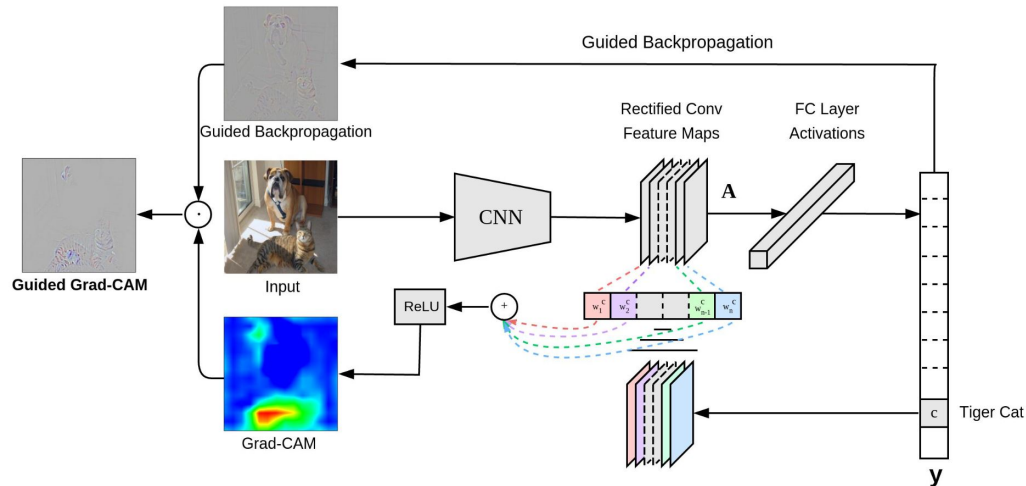
- вход - изображение `image`
- `image.gradient_requires = True`
- делаем forward pass
- делаем backward pass
- возвращаем значения производных по пикселям

Grad-CAM

[paper](#)

Алгоритм:

- вход - изображение image
- делаем forward pass
- сохраняем выход с последнего сверточного слоя (activation maps)
- делаем backward pass до последнего сверточного слоя
- считаем усредненные производные по элементам каналов этого слоя
- считаем сумму activation maps, взвешенную значениями градиентов по каналам
- апсемплим до размера исходного изображения
- возвращаем результат



TCAV

TCAV – Testing with CAVs (2017) <https://arxiv.org/abs/1711.11279>

CAV - Concept Activation Vector

Суть: определяем чувствительность предсказания класса к концепции

Концепция (concept) - осмысленный для человека признак (напр.,
полосатость)

TCAV

Место в таксономии:

- глобальный метод
- не feature-based – объясняет предсказание целиком
- не pixel-wise – объясняет целую картинку

TCAV

Алгоритм для модели $g(x)=h(f(x))$:

1. находим CAV (направление концепции в латентном пространстве)
2. считаем процент сэмплов класса с CAV-Score > 0

TCAV

Алгоритм нахождения CAV:

1. собрать бинарный датасет
 - 1 - картинки, содержащие концепцию
 - 0 - картинки, не содержащие концепцию
2. вычисляем латентные представления этих картинок (activation vectors)
3. обучаем линейный классификатор $wx+b=0$
4. $CAV=w$

TCAV

CAV-Score - чувствительность предсказания класса c на конкретном сэмпле x к концепции

Как считаем: производная по направлению CAV

$$\nabla h(f(x)) \cdot w$$

TCAV

Недостатки

1. не работает с линейно не разделимыми фичами
2. => плохо справляется с shallow NNs

Спасибо за внимание!